



UNIVERSITATEA TEHNICA

DIN CLUJ-NAPOCA

CENTRUL UNIVERSITAR NORD DIN BAIA MARE
FACULTATEA DE INGINERIE

DEPARTAMENTUL DE INGINERIE ELECTRICĂ, ELECTRONICĂ ȘI CALCULATOARE
PROGRAMUL DE STUDII CALCULATOARE

CSWATCH - APLICAȚIE PENTRU GESTIONAREA ȘI URMĂRIREA TURNEELOR DE COUNTER-STRIKE

PROIECT PROIECTAREA APLICAȚIILOR WEB

Student: **Eduard HAIDU**

2024

Cuprins

Capitolul 1	Introducere	1
1.1	Context general	1
1.2	Obiectivele proiectului	1
1.3	Analiza MoSCoW	1
1.4	Cazuri de utilizare	2
Capitolul 2	Arhitectura Aplicației	3
2.1	Front-End	3
2.2	Back-End	3
2.3	Baza de Date	3
Capitolul 3	Implementare frontend	4
3.1	Tehnologii utilizate	4
3.2	Structura interfeței	4
3.2.1	Componente principale	4
3.2.2	Pagini esențiale	5
Capitolul 4	Implementare Backend	7
4.1	Spring Boot	7
4.1.1	Arhitectura Backend	7
4.2	Concepte importante	7
4.2.1	Java Persistence API (JPA)	7
4.2.2	Dependency Injection	8
4.2.3	Gestionarea Cererilor HTTP	9
4.2.4	Elemente de securitate	10
4.2.5	WebSockets	10
4.2.6	Liquipedia API	10
4.3	PostgreSQL	10
Capitolul 5	Concluzii	11

Capitolul 1. Introducere

1.1. Context general

Counter-Strike se numără printre cele mai populare sporturi electronice, iar scena competitivă a acestui joc se află în continuă creștere. Cu timpul, turneele au început să se organizeze tot mai frecvent, iar din ce în ce mai mulți jucători încep să se remarcă. Din acest motiv, fanilor de rând le este mai greu să țină pasul cu toate turneele, schimbările de jucători și echipele. Astfel, a apărut nevoia de a realiza această aplicație, care are ca scop principal centralizarea și urmărirea turneelor de Counter-Strike într-un mod cât mai interactiv și accesibil.

1.2. Obiectivele proiectului

În realizarea acestei aplicații web, ne-am propus să atingem următoarele obiective:

- Utilizatorii să poată vizualiza datele despre turnee, echipe și jucători
- Administratorii să poată adăuga, modifica și șterge turnee și echipe.
- Administratorii să poată modifica rolul și statutul jucătorilor din fiecare echipă
- Administratorii să poată actualiza rezultatele turneelor.

1.3. Analiza MoSCoW

Pentru a determina care cerințe sunt cele mai importante în cadrul aplicației și pentru a le prioritiza mai eficient, am efectuat o analiză MoSCoW pentru a avea o structură clară care ne ajută să categorizăm cerințele în funcție de importanța lor.

Must-have

- Autentificarea administratorilor (Login/Register)
- Gestionare turnee
- Gestionare meciuri
- Vizualizare detalii echipe
- Vizualizare detalii jucători
- Vizualizare meciuri
- Acces diferențiat pe roluri (pentru administratori și utilizatori)

Should-have

- Vizualizare bracket-uri
- Istoric meciuri per echipă
- Statisticile jucătorilor per meci
- Sistem de căutare pentru turnee, echipe, jucători

Could-have

- Sincronizare în timp real a scorurilor
- Interfața intuitivă și instrucțiuni clare
- Securitate avansată
- Clasamente
- Comentarii/Chatting per meci

Won't-have

- Generarea automată a bracket-urilor
- Sistem de pariuri și predicții
- Integrarea cu servicii de streaming (YouTube/Twitch)

1.4. Cazuri de utilizare

Această aplicație este destinată atât jucătorilor de Counter-Strike pasionați de scena competitivă, cât și administratorilor aplicației. În cadrul aplicației am identificat diverse cazuri de utilizare care reflectă scenariile principale pentru utilizatorii noștri. Aceste cazuri de utilizare ilustrează modul în care atât administratorii (Admin), cât și utilizatorii obișnuiți (Guest) interacționează cu aplicația, evidențiind funcționalitățile esențiale și oferind o viziune detaliată asupra experienței utilizatorului. Aceste cazuri de utilizare sunt evidențiate în următoarea diagramă:

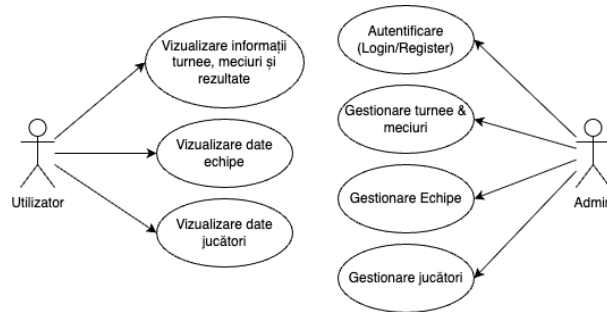


Figura 1.1: Diagrama Cazurilor de utilizare

Capitolul 2. Arhitectura Aplicației

Aplicația a fost realizată cu tehnologii de actualitate pentru a asigura o experiență cât mai plăcută pentru utilizatori, o interfață cât mai prietenoasă și funcționare fără întârzieri sau blocaje. În cadrul acestei aplicații, Front-End-ul a fost dezvoltat în ReactJS, pentru Back-End s-a folosit Spring Boot, iar datele au fost stocate într-o bază de date PostgreSQL.

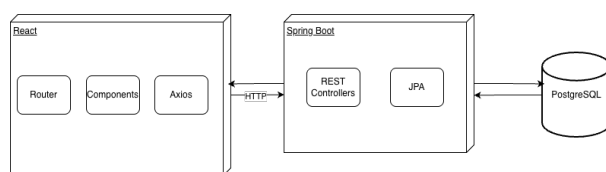


Figura 2.1: Arhitectura Generală

2.1. Front-End

Pentru Front-End am folosit **ReactJS**. React a fost ales pentru partea de front-end datorită modularității, flexibilității și vitezei sale de dezvoltare. Componentele React permit o gestionare eficientă a stării aplicației și facilitarea interacțiunii fluide cu utilizatorul. Structura bazată pe componente oferă un cod ușor de întreținut și extins.

2.2. Back-End

Pentru Back-End am folosit **Spring Boot**. Spring Boot reprezintă coloana vertebrală a back-end-ului, furnizând un cadru robust și ușor de utilizat pentru dezvoltarea serviciilor. Utilizând Spring JPA, interacțiunea cu baza de date PostgreSQL devine transparentă și eficientă. Spring Boot oferă, de asemenea, facilități pentru gestionarea dependențelor, configurării, și dezvoltarea rapidă a serviciilor RESTful.

2.3. Baza de Date

PostgreSQL a fost ales ca sistem de gestionare a bazelor de date datorită robusteții sale, scalabilității și suportului pentru caracteristici avansate. Interfața Spring JPA simplifică interacțiunea cu baza de date, oferind o abordare elegantă pentru manipularea datelor într-un mod eficient și securizat.

Capitolul 3. Implementare frontend

În implementarea interfeței am prioritizat oferirea unei experiențe intuitive și plăcute pentru utilizatori în cadrul aplicației. Obiectivul a influențat alegerea tehnologiilor potrivite.

3.1. Tehnologii utilizate

Motivul pentru care am ales React ca și bibliotecă principală pentru implementarea front-end-ului se datorează posibilității de dezvoltare a componentelor reutilizabile, oferind codului o structură flexibilă și ușor de întreținut. De asemenea, React este o bibliotecă bine documentată ce beneficiază de actualizări regulate și suport din partea comunității. React a permis utilizarea limbajelor Web standard, mai exact: HTML și JavaScript, pentru structura și funcționalitatea paginilor, respectiv CSS cu framework-ul Bootstrap pentru aspectul acestora.

3.2. Structura interfeței

Interfața este organizată în mai multe pagini, care implementează în cadrul acestora diferite componente reutilizabile. Acest lucru reduce foarte mult repetarea codului și facilitează întreținerea și scalarea.

3.2.1. Componente principale

AuthProvider

Componenta wrapper care gestionează starea de autentificare global:

- Verifică token-ul la pornirea aplicației
- Oferă metodele login și logout
- Protejează rutele admin

TopBar

Bara de navigație principală cu link-uri către:

- Home
- Evenimente
- Echipe
- Calendar
- Meniul dedicat pentru admin (dacă un admin este autentificat)
- Autentificare/Înregistrare

React-Brackets

Se folosește biblioteca **react-brackets** pentru afișarea interactivă a bracket-urilor pentru fiecare turneu.

React-Router

Am folosit **React-Router** pentru a împărți paginile pe diferite rute și pentru ca interfața să poată schimba rapid și eficient conținutul paginii în funcție de rută.

3.2.2. Pagini esențiale

Tabela 3.1: Paginile principale din frontend și rolul acestora

Pagină	Rol/Descriere
Home	Pagina principală care afișează primele 6 evenimente viitoare (upcoming) sau în desfășurare, oferind o prezentare generală a platformei
EventsList	Listează toate evenimentele disponibile în baza de date, cu informații despre nume, date, locație și bannere
EventPage	Afișează detaliile complete ale unui eveniment specific, incluzând meciurile asociate și informațiile despre prize pool
EventCalendar	Prezintă evenimentele într-un format de calendar pentru o vizualizare cronologică intuitivă
TeamsList	Afișează toate echipele înregistrate împreună cu membrii lor și logo-urile echipelor
TeamProfile	Prezintă profilul detaliat al unei echipe, incluzând membri, logo, alias și statistici
PlayerProfile	Afișează informații complete despre un jucător: nickname, nume, țară, rol, poză, linkuri către Steam, Faceit și Twitter
Login	Pagină de autentificare pentru utilizatori, gestionând accesul la platformă
Register	Permite înregistrarea utilizatorilor noi în sistem
Logout	Gestionează procesul de deconectare și șterge sesiunea utilizatorului
Liquipedia	Permite realizarea de apeluri către API-ul Liquipedia pentru a obține informații despre turnee, echipe, jucători.
Admin	Dashboard-ul administratorului care oferă acces rapid la funcțiile de management
EventsManager	Interfață administrativă pentru crearea, editarea și ștergerea evenimentelor
MatchesManager	Permite administratorilor să gestioneze meciurile: creare, editare, ștergere și asociere cu evenimente
TeamsManager	Gestionarea echipelor: adăugare, editare, ștergere și încărcare logo-uri
PlayersManager	Administrarea jucătorilor: CRUD complet pentru profilele jucătorilor și asocierea acestora cu echipe

Capitolul 4. Implementare Backend

4.1. Spring Boot

Pentru implementarea părții de backend a aplicației, a fost folosit framework-ul Spring Boot. Spring Boot este un framework Java care facilitează dezvoltarea rapidă a aplicațiilor Java. Alegerea acestuia se datorează modularității sale, capacității de a gestiona dependențe, și abordării convention-over-configuration, care aduce un nivel înalt de consistență în proiect.

4.1.1. Arhitectura Backend

Pe partea de backend, sistemul implementează arhitectura **Model Repository Service Controller**, care oferă următoarele beneficii:

- **Separarea responsabilităților:** Logica de business, modelele de interacțiune cu baza de date și controlerele sunt separate
- **Posibilitate de reutilizare:** Serviciile pot fi injectate în mai multe controlere
- **Gestionarea Tranzacțiilor:** Metodele sunt implicit tranzacționale
- **Testare:** Decuplarea componentelor permite o testare mai ușoară a acestora.

4.2. Concepte importante

4.2.1. Java Persistence API (JPA)

Un avantaj semnificativ al utilizării Spring Boot este opțiunea de a integra **Java Persistence API (JPA)** în cadrul proiectului. JPA este un modul ORM (Object-Relational Mapping) care funcționează ca un punct de legătură între limbajul Java, care este un limbaj orientat pe obiecte, și o bază de date relațională. Practic, JPA ne permite să interacționăm cu baza de date folosind clase Java, fără a scrie instrucțiuni SQL. În cadrul

proiectului s-au implementat cu ajutorul JPA următoarele module importante:

- **Entități (Entities):** Acestea descriu efectiv formatul unei tabele din baza de date

Listing 4.1: Exemplu entitate Player.java

```
@Entity
@Table(name = "players")
public class Player {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "nickname", nullable = false)
    private String nickname;

    @Column(name = "first_name")
```

```

    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "country")
    private String country;

    @Column(name = "date_of_birth")
    private LocalDate dateOfBirth;

    @Column(name = "team_id")
    private Integer teamId;

    @Column(name = "role")
    private String role;

    @Column(name = "photo_path")
    private String photoPath;
}

```

- **Repositories:** Acestea permit realizarea operațiilor CRUD (Create, Read, Update, Delete) cu ajutorul entităților

Listing 4.2: PlayerRepository.java

```

@Repository
public interface PlayerRepository
    extends JpaRepository<Player, Integer> {

    List<Player> findByTeamId(Integer teamId);
}

```

Principalele avantaje ale utilizării JPA sunt:

- **Abordare Obiect-Relațională:** JPA permite lucrul cu obiecte Java în locul interogărilor SQL directe, oferind o abordare mai orientată pe obiect în gestionarea datelor.
- **Reutilizabilitate și consistență:** Utilizarea entităților JPA și a repository-urilor abstractizează detaliile tehnice ale manipulării datelor, facilitând reutilizarea și menținerea consistenței.

4.2.2. Dependency Injection

Implementarea backend-ului folosind Spring Boot ne permite să folosim **Dependency Injection**, un concept fundamental în programarea orientată pe obiecte, unde obiectele nu își instanțiază direct dependențele, ci le primesc din exterior. Implementarea Dependency Injection are următoarele avantaje:

- **Decuplare și testare ușoară:** Dependency Injection promovează decuplarea între componente, facilitând testarea unitară și schimbul simplu al implementărilor.
- **Gestionarea ciclurilor de viață:** Spring se ocupă de gestionarea ciclurilor de viață ale componentelor, asigurând crearea și distrugerea corespunzătoare a acestora.

- **Configurare ușoară:** Configurarea dependențelor se face în mod declarativ, adesea prin intermediul XML sau a notărilor, facilitând gestionarea și modificarea dependențelor.

4.2.3. Gestionarea Cererilor HTTP

Întreaga aplicație funcționează pe baza comunicării dintre frontend și backend prin intermediul cererilor HTTP. Este importantă existența endpoint-urilor bine definite, cu un nume concis, pentru a urmări logica aplicației și fluxul de comunicare. Endpoint-urile importante pentru aplicație sunt următoarele:

Endpoint-uri pentru jucători

Metodă	Endpoint	Descriere
GET	/players/all	Obține toți jucătorii
GET	/players/{id}	Obține jucătorul după ID
POST	/players	Crează jucător nou
PUT	/players/{id}	Actualizează jucătorul
DELETE	/players/{id}	Șterge jucătorul
GET	/players/team/{teamId}	Obține jucătorii după ID-ul echipei

Endpoint-uri pentru echipe

Metodă	Endpoint	Descriere
GET	/teams/all	Obține toate echipele
GET	/teams/{id}	Obține echipa după ID
POST	/teams	Crează o echipă nouă
PUT	/teams/{id}	Actualizează echipă
DELETE	/teams/{id}	Șterge echipă

Endpoint-uri pentru turnee

Metodă	Endpoint	Descriere
GET	/events/all	Obține toate turneele
GET	/events/{id}	Obține turneul după ID
POST	/events	Crează turneu nou
PUT	/events/{id}	Actualizează turneu
DELETE	/events/{id}	Șterge turneu

Endpoint-uri pentru hărți

Metodă	Endpoint	Descriere
GET	/matches/tournament/{id}	Obține meciurile după turneu
POST	/matches	Crează un meci nou
PUT	/matches/{id}	Actualizează meci
DELETE	/matches/{id}	Șterge meci

4.2.4. Elemente de securitate

Pentru a gestiona în mod eficient și sigur autentificarea utilizatorilor și autorizarea pentru acces la endpoint-uri, am folosit **Json Web Token (JWT)**. În momentul în care un utilizator se autentifică cu succes, se generează un token JWT care conține numele utilizatorului și drepturile sale de acces. Acest token este trimis împreună cu cererile HTTP pentru a avea acces la endpoint-urile protejate. Avantaje:

- **Stateless și Eficiență:** JWT permite aplicației să fie stateless, eliminând necesitatea de a menține sesiuni la nivel de server. Token-ul conține toate informațiile necesare, evitând interacțiunile constante cu o bază de date pentru validare.
- **Flexibilitate și Siguranță:** JWT oferă flexibilitate prin includerea unui payload semnat digital, asigurând integritatea și autenticitatea informațiilor.

4.2.5. WebSockets

Pentru actualizarea scorurilor în cadrul meciurilor în timp real și pentru trimiterea notificărilor, am folosit **WebSockets**. Această tehnologie permite transmiterea instantanee a informațiilor, oferind o experiență mai interactivă și mai plăcută.

4.2.6. Liquipedia API

Pentru extragerea informațiilor de ultimă oră despre turnee, echipe și jucători, administratorii pot să facă apeluri către API-ul Liquipedia. Liquipedia este o bază de date extinsă care conține toate informațiile despre Counter Strike competitiv și este actualizată constant, cu suport din partea comunității și un API bine documentat. Administratorii au nevoie de URL-ul paginii Liquipedia din care doresc să extragă informații. Apoi, backend-ul nostru primește un răspuns de la API sub formă de **Wiki Markup**. Pentru a extrage informația dorită din răspunsul oferit de API, sunt necesare efectuarea a mai multor operații de căutare folosind pattern-uri **Regular Expression (RegEx)** și convertirea informației la modelul bazei de date.

4.3. PostgreSQL

Am optat pentru PostgreSQL ca și sistem de gestionare a bazelor de date, datorită stabilității, performanței, și funcționalităților avansate pe care le oferă. PostgreSQL suportă tipuri de date complexe, tranzacții ACID, și este recunoscut pentru gestionarea eficientă a volumelor mari de date.

Capitolul 5. Concluzii

Implementarea platformei de gestionare și urmărire a turneelor de Counter Strike a reprezentat dezvoltarea unei soluții software complete (full-stack) pentru gestionarea ecosistemului competițional de Counter-Strike 2. Prin îmbinarea robusteții framework-ului Spring Boot pe partea de backend cu flexibilitatea bibliotecii React pe frontend, s-a obținut o aplicație scalabilă, securizată și ușor de utilizat.

Principalele obiective atinse în cadrul acestui proiect includ:

- **Arhitectură Decuplată:** Implementarea unei arhitecturi de tip RESTful API a permis o separare clară între logica de business (Java) și interfața utilizatorului (TypeScript). Aceasta facilitează întreținerea codului și permite dezvoltarea independentă a celor două componente.
- **Managementul Complex al Datelor:** Utilizarea Spring Data JPA și MySQL a permis modelarea eficientă a relațiilor complexe specifice domeniului eSports (ex: relația many-to-one dintre jucători și echipe, istoricul transferurilor, structura arborescentă a bracket-urilor de turneu).
- **Securitate și Control:** Integrarea Spring Security cu autentificarea bazată pe JWT (JSON Web Tokens) a asigurat protecția datelor sensibile și a permis implementarea unui sistem granular de permisiuni (roluri de Admin vs. Public User), esențial pentru integritatea datelor competiționale.
- **Experiența Utilizatorului (UX):** Interfața modernă, realizată cu React 18 și CSS3, oferă o navigare intuitivă. Vizualizarea dinamică a bracket-urilor de turneu (folosind react-brackets) și designul responsiv (mobile-friendly) rezolvă nevoia comunității de a accesa rapid informații despre meciuri și scoruri de pe orice dispozitiv.
- **Integrare Externă:** Capacitatea de a prelua date din surse externe (Liquipedia) demonstrează flexibilitatea platformei de a nu opera într-un sistem închis, ci de a se sincroniza cu ecosistemul global de CS2.

În concluzie, aplicația reușește să centralizeze fluxul operațional al organizatorilor de turnee, eliminând necesitatea gestionării manuale (ex: foi de calcul) și oferind fanilor o sursă centralizată de informație.